

# Architectural Blueprint for a Sovereign, Mobile-Native Layer-1 Ride-Sharing Network

## 1. Analysis of Mobile-Native Layer-1 Precedents

The pursuit of deploying a fully sovereign, decentralized physical infrastructure network (DePIN) entirely on mobile devices represents one of the most formidable challenges in the field of distributed systems. A sovereign Layer-1 (L1) blockchain demands that participating nodes execute consensus algorithms, maintain the ledger state, and process transactions locally without deferring to centralized cloud servers, remote procedure call (RPC) providers, or dependent Layer-2 rollup architectures.<sup>1</sup> To construct an independent, middleman-free ride-sharing network where standard consumer smartphones act as the sole infrastructure, it is imperative to deeply dissect the architectural precedents that have successfully embedded blockchain protocols into resource-constrained edge devices. Three primary precedents dictate the current state-of-the-art in mobile-native decentralized systems: Minima, the Mina Protocol, and the Nodle Network.

Minima provides a foundational blueprint for edge consensus and state pruning, engineered specifically to operate as a completely decentralized network where every smartphone runs a fully validating and constructing node.<sup>3</sup> To overcome the physical limitations of mobile storage, processing power, and battery constraints, Minima utilizes a cooperative Proof-of-Work (Tx-PoW) mechanism.<sup>4</sup> Instead of nodes competing via computationally intense, brute-force hashing—which would rapidly degrade smartphone hardware and deplete battery life—nodes collaborate by performing a minor cryptographic computation to process their own transactions before forwarding them to peers.<sup>5</sup> This cooperative approach aggregates fractional computations into a secure network consensus while capping mobile energy demand to less than two percent of daily battery use.<sup>7</sup> Furthermore, Minima addresses state bloat through an innovative "Cascading Chain" architecture. The network prunes historical data logarithmically using "Superblocks," which maintain cryptographic proof of cumulative computational work without storing the underlying transactional data of obsolete blocks.<sup>6</sup> The Cascading Chain consists of 32 hierarchical levels, holding a maximum of 128 blocks at each level, ensuring the chain grows logarithmically rather than linearly.<sup>6</sup> The state itself is managed via a Merkle Mountain Range (MMR) database, ensuring nodes only retain Unspent Transaction Outputs (UTXOs) relevant to their specific private keys, thereby keeping local storage requirements below 1GB and memory utilization around 300MB.<sup>5</sup>

While Minima solves storage constraints through selective database pruning, the Mina Protocol takes a purely cryptographic approach, maintaining a fixed blockchain size of approximately 22KB regardless of transaction volume.<sup>8</sup> Mina achieves this extreme compression by entirely

replacing the conventional block history with a singular, easily verifiable cryptographic proof.<sup>12</sup> Utilizing a recursive Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zk-SNARK) structure—specifically the Pickles SNARK—Mina nodes do not need to download the ledger history.<sup>8</sup> Instead, every time a new block is produced, a zk-SNARK is generated that proves the validity of the new block and recursively verifies the validity of the previous SNARK.<sup>15</sup> This recursive composition effectively compresses the entire computational history of the network into a constant-size certificate, paired with a tail-end Merkle path to prove current account balances.<sup>15</sup> This architectural decision eliminates the reliance on heavyweight infrastructure, allowing any mobile device to achieve full-node security guarantees with sub-second synchronization times directly within a browser or native application.<sup>8</sup> Mina relies on an Ouroboros Samasika Proof-of-Stake consensus algorithm, specifically optimized for recursive composition to minimize computational overhead during verification.<sup>17</sup>

The Nodle Network diverges from general-purpose computation to focus heavily on decentralized physical infrastructure, utilizing smartphones as mobile base stations to relay data from Bluetooth Low Energy (BLE) devices to the blockchain.<sup>19</sup> Nodle bypasses computationally heavy consensus on the mobile device by employing a "Proof of Connectivity" (PoC) algorithm.<sup>21</sup> Smartphones act as edge nodes, earning native tokens based on network bandwidth provision, geographic coverage, and the density of localized connections.<sup>22</sup> While Nodle currently operates atop zkSync Era (an Ethereum Layer-2 scaling solution) rather than as a standalone sovereign mobile L1, its validation of the "Smartphone Infrastructure" model proves that battery-efficient, background BLE mesh networking is highly viable at a global, commercial scale.<sup>21</sup>

Conversely, negative precedents such as Pi Network serve as cautionary tales against "decentralization theater." Pi Network presents itself as a mobile-mining ecosystem but functions primarily as a lightweight client relying on centralized, invitation-only trust graphs modeled after the Stellar Consensus Protocol.<sup>26</sup> Users essentially interact with a centralized database that mints tokens based on daily engagement, lacking true sovereign execution or cryptographic consensus.<sup>27</sup> True sovereignty requires moving beyond layered architectures, where rollups depend on base layers, bridges depend on relayers, and execution relies on centralized RPC nodes; these dependencies create fragmented liquidity, increased attack surfaces, and a fundamental loss of control.<sup>2</sup>

| Architecture Parameter | Minima                | Mina Protocol       | Nodle Network             | Proposed Ride-Sharing L1   |
|------------------------|-----------------------|---------------------|---------------------------|----------------------------|
| Storage Solution       | Cascading Chain / MMR | Recursive zk-SNARKs | External L1/L2 Dependency | Block-Lattice + IVC SNARKs |

|                            |                       |                          |                       |                                            |
|----------------------------|-----------------------|--------------------------|-----------------------|--------------------------------------------|
| <b>Consensus Mechanism</b> | Collaborative Tx-PoW  | Ouroboros Samasika (PoS) | Proof of Connectivity | Spatial PoC + TEE Proof-of-Location        |
| <b>P2P Networking</b>      | Maxima (IP-based P2P) | Standard Gossip Protocol | BLE Edge Nodes        | BLE / Wi-Fi Aware + Distributed Hash Table |
| <b>Ledger State Size</b>   | ~1GB (Pruned)         | ~22KB (Constant)         | N/A (Light Client)    | Constant size (~50KB)                      |

The synthesis of these precedents dictates that a sovereign ride-sharing architecture must combine Minima's collaborative edge-computation and device-level autonomy, Mina's recursive zero-knowledge proofs for absolute state compression, and Nodle's localized hardware telemetry (BLE/Wi-Fi) for physical world connectivity.

## 2. Solving Historical Architectural Failures in Decentralized Ride-Sharing

The conceptual framework of decentralized ride-sharing has been attempted previously by platforms such as Arcade City, La'Zooz, Eva, and Teleport.<sup>28</sup> Despite their ideological ambitions to displace corporate monopolies, these platforms largely failed to capture mainstream market share or maintain technological resilience. Designing a truly sovereign L1 requires a ruthless deconstruction of why these predecessors failed, ranging from poor unit economics to devastating single points of failure.

Early decentralized platforms fell into the fatal trap of attempting to emulate centralized Web2 operations without possessing the venture capital to subsidize the supply and demand sides of the marketplace, a dynamic known as the "Uber for X" unit economics trap.<sup>32</sup> Furthermore, these protocols suffered from catastrophic infrastructural economics because they built their platforms as smart contracts on top of existing, congested Layer-1 networks like Ethereum. The inherent latency of standard blockchain block times combined with unpredictable, fluctuating gas fees made executing real-time dispatch and micro-payments economically irrational.<sup>33</sup> A rider cannot practically wait several minutes for block finality and pay high transaction fees to secure a low-cost, short-distance ride. This underlying architectural flaw forced early networks to compromise on their decentralized ethos.

Because these early architectures could not handle real-time geospatial matching on-chain, they deferred to centralized off-chain databases and traditional Web2 communication layers.

Arcade City, for instance, relied heavily on closed Facebook groups and centralized communication APIs to facilitate ride requests and coordinate dispatch.<sup>32</sup> This hybrid approach entirely negated the security and autonomy benefits of blockchain technology, introducing critical single points of failure, compromising user privacy, and subjecting the network to the same regulatory vulnerabilities as centralized applications.<sup>28</sup> Relying on external servers means that if the server farm goes offline, the API provider alters its terms of service, or a centralized indexer fails, the entire ride-sharing network collapses.<sup>2</sup>

Moreover, without a centralized corporate entity to ban bad actors or process refunds, early decentralized ride-sharing failed to establish algorithmic trust. They suffered from a severe Sybil deficit, meaning malicious actors could trivially spin up thousands of fake driver or rider nodes to manipulate network routing, cancel rides in bulk, or artificially inflate regional demand to trigger surge pricing.<sup>35</sup> Furthermore, without cryptographic, zero-knowledge location proofs, drivers could spoof their GPS coordinates at the operating system level to collect geospatial rewards without physically moving, completely destroying the integrity of the dispatch system.

To prevent repeating these historical failures, the proposed sovereign L1 architecture establishes three non-negotiable design principles. First, the network mandates zero dependency on cloud RPCs; all communications, matching, and data storage must occur peer-to-peer using localized smartphone hardware. Second, the architecture demands the decoupling of execution from global settlement. Real-time matching and geospatial querying must never wait for global blockchain consensus; they must execute on an ephemeral, localized data layer, while the blockchain is strictly reserved for asynchronous financial settlement and dispute resolution. Finally, the network must employ algorithmic dispute resolution driven by cryptographic hardware attestation to mathematically guarantee location and identity, thereby automating conflict management without requiring a centralized human helpdesk.

### **3. Sub-Second Matching and Storage Bloat Mitigation**

To achieve sub-second matching for ride-sharing while running a full node exclusively on consumer smartphones, the architecture must completely abandon the monolithic blockchain paradigm. Traditional blockchains force every node to sequence, execute, and store every transaction globally, creating an ever-growing storage burden that eventually precludes mobile users from participating as full nodes.<sup>38</sup> To mitigate storage bloat and eliminate transaction bottlenecks, the network must adopt a directed acyclic graph (DAG) variant known as the Block-Lattice, combined with advanced stateless execution proofs.

In a block-lattice structure, pioneered by protocols like Nano, the concept of a singular, shared global ledger is discarded in favor of a highly parallelized architecture.<sup>39</sup> Every user, whether a driver or a rider, maintains their own independent blockchain, referred to as an account-chain.<sup>41</sup> When a rider initiates a payment, they append a "Send" block to their personal chain, which subtracts the balance and includes a reference to the previous block. Concurrently, the driver

appends a matching "Receive" block to their personal chain to update their balance.<sup>42</sup> Because these chains operate asynchronously, users can update their account states instantly without waiting for a global network block to be mined, removing the primary bottleneck for transaction throughput.<sup>40</sup> Crucially, the block-lattice allows for extreme ledger pruning. Because each transaction block contains the complete current state and balance of the account, the historical sequence of transactions becomes cryptographically irrelevant for verifying current liquidity.<sup>43</sup> The protocol can automatically prune the database down to a single "State Block" per user account, dramatically reducing long-term storage requirements.<sup>44</sup>

While the block-lattice efficiently manages individual accounts, the network still requires a mechanism for nodes to reach consensus on the global state root without storing the entire dataset. The architecture resolves this by incorporating Verkle Trees, a highly efficient data structure that serves as an evolution of the Merkle Patricia Tree by utilizing Vector Commitments.<sup>47</sup> Verkle Trees drastically reduce the size of cryptographic witnesses required to prove the inclusion of data within a block.<sup>48</sup> In a stateless execution model, a smartphone node does not need to store the global ledger. When it receives a block, the block is accompanied by a compact Verkle multiproof containing only the specific state data necessary to execute the transactions within that block.<sup>47</sup> Once the node verifies the proof against the global state root, it discards the data, operating entirely on-demand and keeping persistent disk usage virtually non-existent.<sup>49</sup> While researchers have identified potential deserialization bottlenecks and memory copying redundancies in early Verkle implementations, optimizing the deserialization logic allows Verkle trees to offer constant, fast verification times with proof sizes on the order of just one megabyte, making them ideal for mobile deployment.<sup>48</sup>

To achieve true succinctness akin to the Mina Protocol, the network must continuously compress the history of the block-lattice using recursive zk-SNARKs.<sup>38</sup> However, generating traditional recursive SNARKs is computationally intensive and would rapidly deplete mobile battery life. The optimal solution lies in advanced cryptographic folding schemes, specifically Nova, SuperNova, and Sangria.<sup>38</sup> Nova provides highly efficient Incrementally Verifiable Computation (IVC).<sup>56</sup> Instead of executing a heavy verifier circuit inside a SNARK at every step of computation—which exponentially increases prover overhead—Nova utilizes a folding scheme to compress two instances of a Rank-1 Constraint System (R1CS) into a single relaxed R1CS instance.<sup>58</sup> Mathematically, a prover takes a running instance and a new computational step, folding them together through simple hashing and scalar multiplications, completely bypassing the need to generate a full SNARK during intermediate steps.<sup>38</sup> A heavy SNARK is only generated at the very end of the lifecycle (e.g., when a driver closes their operational shift) to prove the validity of the final folded instance.<sup>38</sup> SuperNova extends this capability by supporting Non-Uniform IVC (NIVC), allowing the network to fold different types of smart contract executions—such as ride matching, escrow locking, and penalty slashing—without incurring performance penalties, while Sangria adapts these folding techniques for PLONK arithmetization.<sup>38</sup>

Even with a pruned block-lattice and IVC folding schemes, committing every micro-interaction

to the Layer-1 ledger is inefficient. Sub-second matching and continuous micro-payments, such as paying a driver per mile driven, require the integration of Layer-2 State Channels.<sup>2</sup> When a rider and driver match, their smartphones open an encrypted, direct peer-to-peer state channel. The rider locks the estimated fare in a multi-signature escrow on the L1 block-lattice. As the vehicle physically moves, the rider's phone streams micropayment signatures to the driver's phone locally over Bluetooth or Wi-Fi Direct. These off-chain transactions execute in milliseconds with zero network fees.<sup>7</sup> Upon arrival at the destination, the channel closes, and the final state is cryptographically folded via Nova and broadcasted to the L1 block-lattice for asynchronous settlement, ensuring the L1 is never congested by the granular telemetry of the ride.<sup>54</sup>

## 4. Sybil Resistance and Hardware Attestation (Proof of Location)

In a decentralized ride-sharing network completely devoid of central authorities, verifying that a driver or rider is a unique human being located at a specific geographic coordinate is a foundational security requirement. Software-based geolocation is fundamentally insecure; an attacker can easily spoof GPS coordinates at the operating system level to manipulate vehicle routing, claim false rides, or execute a devastating Sybil attack by masquerading as thousands of distinct vehicles to artificially trigger supply-demand surge pricing.<sup>35</sup> To achieve uncompromisable Sybil resistance, the L1 architecture bypasses the software layer entirely, leveraging the specialized security hardware natively embedded in modern smartphones: Trusted Execution Environments (TEEs).<sup>63</sup>

A Trusted Execution Environment is a physically and logically isolated secure subsystem within the smartphone's main processor.<sup>65</sup> Platforms such as ARM TrustZone on Android devices and the Apple Secure Enclave on iOS operate entirely independently of the Rich Execution Environment (REE) or main operating system.<sup>66</sup> This isolation guarantees that even if the host operating system is rooted, jailbroken, or infected with kernel-level malware, the TEE's memory, cryptographic keys, and execution environment remain completely impenetrable.<sup>64</sup> The Apple Secure Enclave, for example, utilizes a dedicated custom L4 microkernel, isolated memory encrypted by a hardware AES engine, and a Boot ROM that establishes an immutable root of trust that cannot be updated or altered by software.<sup>65</sup> Cryptographic keys generated within the TEE can never be extracted or viewed by the host operating system; the OS can only request that the TEE sign a payload, and the TEE will only comply if the request meets strictly defined security parameters.<sup>68</sup>

To secure the ride-sharing network, the architecture implements a cryptographically bound Proof of Location (PoL) protocol that relies exclusively on this hardware isolation.<sup>36</sup> Upon network initialization, the user's smartphone generates an asymmetric keypair strictly inside the TEE.<sup>68</sup> The resulting public key becomes the node's permanent, unforgeable sovereign identity on the block-lattice. To prevent GPS spoofing, baseband data from the smartphone's



GPS chip, cellular triangulation telemetry, and Wi-Fi/BLE proximity sensor arrays are routed directly via secure hardware buses into the TEE, entirely bypassing the malleable host operating system.<sup>66</sup>

The TEE processes this raw sensor data, verifies its authenticity, and executes a zero-knowledge proof (specifically a zk-SNARK) asserting a rigid geographic statement: that the specific hardware module is currently situated at a specific latitude and longitude, with a defined margin of error, at a precise timestamp. The TEE then signs this ZK-proof with the unextractable private key and outputs the signed payload to the main OS for broadcasting to the decentralized network.<sup>66</sup> Because the signature originates from a hardware-attested TEE and is verified via a zk-SNARK, the network can trust the location data with absolute mathematical certainty, without exposing the raw, continuous movement data of the user to the public ledger, thus preserving user privacy.<sup>71</sup>

By tying network participation to this hardware-backed identity, the cost of executing a Sybil attack becomes physically and economically prohibitive.<sup>35</sup> An attacker cannot simulate ten thousand drivers using a virtualized script on a cloud server; they would need to physically purchase ten thousand distinct flagship smartphones, successfully bypass each individual hardware enclave (an exploit that is exceedingly rare and rapidly patched), and physically distribute those devices across a geographic city to mimic real-world traffic patterns.<sup>37</sup> Furthermore, while TEEs share physical resources like power and cache with the normal processor—making them theoretically vulnerable to advanced side-channel attacks like Spectre or Meltdown—the protocol mitigates these hardware vulnerabilities by requiring continuous, rolling cryptographic keys and utilizing biometric commitments.<sup>70</sup> For instance, FaceID or TouchID data, which is processed purely within the Secure Enclave, serves as a secondary validation trigger during ride initiation, ensuring that the human operating the device matches the hardware identity and defeating attempts to automate the devices remotely.<sup>68</sup>

## 5. Serverless P2P Networking and Geospatial Indexing

Centralized platforms like Uber achieve sub-second matching by continuously streaming massive volumes of driver telemetry to centralized databases (such as Redis), utilizing spatial indexing algorithms like GeoHashes or Quadrees to query availability.<sup>77</sup> A sovereign Layer-1 operating exclusively on edge devices must replicate this high-frequency, low-latency geospatial querying without relying on any central index server, load balancers, or message queues like Kafka.<sup>77</sup>

The foundational network layer replaces internet-routing dependencies with localized radio frequencies, taking advantage of capabilities built natively into modern smartphone hardware. Smartphones natively support protocols like Wi-Fi Aware (Wi-Fi Direct) and Bluetooth Low Energy (BLE) Mesh networking.<sup>81</sup> BLE Mesh is highly optimized for low-power, continuous background broadcasting. Driver nodes emit BLE advertisement packets containing their TEE-attested Proof of Location and availability status, while rider nodes passively scan these

frequencies to detect drivers within an immediate physical radius.<sup>22</sup> For extended range and higher bandwidth requirements beyond a localized radius, Wi-Fi Aware allows smartphones to form dynamic, ad-hoc clusters without requiring a local router or active internet connection.<sup>82</sup> This allows nodes to construct a localized mesh, routing ride requests across multiple physical smartphone hops to find matches within a broader neighborhood context, effectively turning urban centers into vast, decentralized communication webs.<sup>82</sup>

While radio-frequency mesh networking handles hyper-localized matching, city-wide dispatch requires a robust, decentralized data structure capable of managing thousands of concurrent queries. The architecture employs a Distributed Hash Table (DHT)—similar to the Kademlia protocol used in BitTorrent—as a decentralized overlay network spanning all active smartphones.<sup>85</sup> However, standard DHTs natively support only exact-match key lookups, which are practically useless for the proximity queries required by ride-sharing networks (e.g., finding the nearest five drivers to a specific coordinate).<sup>80</sup>

To solve the spatial querying dilemma, the network implements a Distributed Spatial Quadtree (DSTree) mapped directly onto the DHT architecture.<sup>87</sup> The geographic map is recursively subdivided into four quadrants (a Quadtree), with each quadrant representing a specific geospatial bounding box.<sup>78</sup> These spatial bounding boxes are mathematically translated into one-dimensional string identifiers using space-filling curves, such as a Z-order or Hilbert curve (creating Geohashes).<sup>89</sup> These resulting Geohash strings are then utilized as routing keys within the DHT.<sup>91</sup> Node IDs, derived from the TEE public keys of participating smartphones, that are numerically close to a specific Geohash take responsibility for storing and routing the telemetry data for that specific physical region.<sup>91</sup>

When a rider inputs a destination and requests a ride, their smartphone calculates the Geohash of their current location and queries the DHT for that specific key.<sup>91</sup> The query is efficiently routed in  $O(\log n)$  hops to the specific peer currently maintaining that spatial shard.<sup>85</sup> Because drivers continuously update their location to the DHT node managing their specific Geohash, the rider's device instantly receives a localized subset of available drivers, achieving sub-second spatial querying entirely via peer-to-peer infrastructure.<sup>77</sup>

To handle the "double booking" problem—where multiple drivers might attempt to accept the same ride request simultaneously—the P2P network employs Distributed Locks and Optimistic Concurrency Control across the localized mesh.<sup>77</sup> When a driver accepts a ride, the local DHT shard checks a version number; the first request successfully updates the ride status, and any subsequent requests within that millisecond are rejected, ensuring strict transactional consistency without a central database.<sup>77</sup>

## 6. Ground-Up Sovereign Architecture Design

Integrating these highly advanced cryptographic, networking, and hardware paradigms yields a highly optimized, four-layer architecture capable of running entirely on consumer smartphones



without degradation in user experience or network reliability.

**Layer 0: The Hardware and Transport Layer** This foundational layer forms the physical root of trust and localized connectivity. It encompasses the smartphone's radio antennas (LTE/5G, BLE, Wi-Fi Aware) alongside the Trusted Execution Environment (ARM TrustZone, Apple Secure Enclave).<sup>63</sup> Layer 0 is strictly responsible for generating true cryptographic entropy, executing hardware-attested Proof of Location (PoL) by ingesting baseband sensor data, and maintaining localized ad-hoc mesh connectivity between proximate devices.<sup>66</sup> This layer ensures that the data fed into the blockchain is mathematically tamper-proof from its inception.

**Layer 1: The Sovereign Block-Lattice (State and Consensus)** The core ledger completely discards the linear blockchain bottleneck, utilizing a block-lattice structure for infinite parallel scalability.<sup>39</sup> Each user maintains a personal sovereign chain consisting only of their most recent state block, which encapsulates their balance and active escrow data, allowing for extreme state pruning that keeps local storage requirements under 50MB.<sup>43</sup> To finalize cross-chain interactions and prevent double-spending, the network employs a highly optimized, asynchronous consensus mechanism.<sup>13</sup> Verkle Trees and Nova IVC folding schemes ensure that when a node boots up, it verifies a constant-size zk-SNARK (e.g., ~22KB) to instantly validate the integrity of the entire global network state without needing to download historical blocks.<sup>8</sup>

**Layer 2: The Ephemeral Indexing Layer (DHT + DSTree)** Layer 2 is non-transactional, entirely ephemeral, and serves exclusively as the geospatial matching engine. It consists of the Kademlia DHT overlay and the DSTree spatial index.<sup>85</sup> Driver telemetries are broadcast to this layer as short-lived, signed packets. This layer is designed to dynamically balance loads; for example, if tens of thousands of users request rides simultaneously after a major sporting event, the quadtree dynamically subdivides the region into smaller, high-resolution geohashes, distributing the query load across hundreds of nearby idle smartphones to prevent localized network crashes and maintain sub-second matching.<sup>77</sup>

**Layer 3: The Execution Layer (State Channels & zkVM)** All complex business logic executes at this layer. Once a rider and driver are matched via Layer 2, they establish a direct, encrypted P2P state channel.<sup>7</sup> A Zero-Knowledge Virtual Machine (zkVM) residing locally on the device executes the smart contract governing the specific terms of the ride.<sup>38</sup> The ride fare is streamed cryptographically per GPS polling interval. Upon reaching the physical destination, the channel closes, and the final state is folded and submitted to Layer 1 for asynchronous financial settlement.<sup>38</sup>

## 7. Native Economics and Game Theory

To function autonomously without a centralized corporate intermediary dictating fares and extracting up to 30% in commissions, the network relies on cryptographically enforced game theory and decentralized economics to align the incentives of riders, drivers, and infrastructure providers.<sup>31</sup>

## Algorithmic, Oracle-Free Dynamic Pricing

Centralized ride-sharing platforms utilize proprietary, closed-source algorithms to calculate "surge" pricing based on black-box supply and demand metrics, often to the detriment of both drivers and riders. The sovereign L1 implements dynamic pricing natively via the DHT and the local zkVM, removing the need for external pricing oracles.<sup>97</sup> When a rider queries the DSTree for nearby drivers, the specific DHT shard returns the localized ratio of active driver pings to rider requests.<sup>88</sup> A deterministic, open-source pricing curve encoded directly into the zkVM takes this ratio, alongside the physical distance and real-time traffic density inferred from vehicle telemetry, to calculate a fair-market fare. Because this calculation occurs locally and is verified by a zk-SNARK, neither party can maliciously alter the fare multiplier to extort the other.<sup>99</sup>

## Deflationary Tokenomics and Burn Mechanics

The network requires a native utility token strictly for securing the ledger, transferring value, and preventing network spam. Instead of extracting a percentage fee to pay a corporate entity, the network relies on a continuous "Burn" mechanism.<sup>3</sup> Every Layer-1 transaction—such as opening a state channel or settling a final balance—requires the initiator's device to perform a small, collaborative proof-of-work computation (Tx-PoW) and permanently burn a fractional amount of the native token.<sup>5</sup> This provides an intrinsic economic cost to network spam. As network usage and ride volume grow globally, the total circulating supply of the token systematically decreases. This applies continuous deflationary pressure, organically appreciating the value of the token held by drivers and active network participants without relying on speculative inflation.<sup>3</sup>

## Incentive Structures for Infrastructure Provision

While riders and drivers are inherently self-interested actors, the ephemeral DHT indexing layer requires mobile devices to expend battery and bandwidth to store and route telemetry data for third parties.<sup>80</sup> To ensure the network remains highly available and resilient, devices that passively act as routing nodes or DSTree shard managers earn micro-rewards. Drawing direct inspiration from Nodle's Proof of Connectivity, idle smartphones that dedicate RAM and background bandwidth to maintaining the geospatial index receive fractional token emissions generated by the protocol, essentially paying the public to maintain the network infrastructure.<sup>19</sup>

## Trustless Dispute Resolution

If a driver fails to drop off a rider at the designated location, or a rider attempts to cancel maliciously mid-ride, there is no centralized customer service line to intervene. Instead, the multi-signature state channel acts as the ultimate, algorithmic arbiter.<sup>7</sup> Both parties are required to stake a small collateral when initiating the ride. The continuous stream of TEE-backed Proof of Location data acts as an objective, cryptographic truth of the vehicle's

movement.<sup>71</sup> If a dispute is raised, the zkVM mathematically evaluates the cryptographic location signatures against the agreed-upon route coordinates. The smart contract programmatically slashes the collateral of the party whose hardware attestation deviated from the contract terms, executing perfect algorithmic justice without human bias.<sup>96</sup>

## 8. Granular Roadmap and Implementation Checklist

Deploying a mobile-native sovereign Layer-1 with localized geospatial indexing requires a phased, rigorous engineering roadmap. The following checklist details the architectural progression from foundational cryptography to user-facing execution, serving as the blueprint for development.

| Implementation Phase                                              | Strategic Objective                                                                        | Key Engineering Milestones                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Phase 1: Cryptographic Primitives and Hardware Abstraction</b> | Establish the unforgeable root of trust and cryptographic succinctness on mobile hardware. | <ul style="list-style-type: none"><li>- <b>Implement Hardware Interfaces:</b> Develop low-level C++/Rust APIs to bridge the mobile OS with ARM TrustZone and Apple Secure Enclave.</li><li>- <b>Establish TEE Key Generation:</b> Ensure elliptic curve keypairs (e.g., secp256k1 or Ed25519) are generated securely within the enclave to establish unextractable node identities.</li><li>- <b>Develop PoL Oracles:</b> Route baseband GPS, BLE, and cellular triangulation data directly into the TEE. Write the microkernel logic to sign this data and output localized zero-knowledge proofs.</li><li>- <b>Integrate Folding Schemes:</b> Implement a Rust-based Nova/SuperNova library for Incrementally Verifiable</li></ul> |

|                                                                |                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                |                                                                  | Computation (IVC), enabling R1CS folding on ARM-based mobile processors to achieve constant-size proofs.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Phase 2: Peer-to-Peer Networking and Geospatial Overlay</b> | Create the serverless communication and spatial matching engine. | <p>- <b>Deploy Radio Mesh Networking:</b> Write device-specific modules for BLE advertisement broadcasting and Wi-Fi Aware service discovery to handle localized peer handshakes.</p> <p>- <b>Construct the Kademlia DHT:</b> Build the decentralized hash table overlay tailored for high-churn mobile environments, utilizing UDP hole punching for NAT traversal.</p> <p>- <b>Map the DSTree Spatial Index:</b> Implement the Distributed Spatial Quadtree. Encode geospatial bounding boxes into 1D space-filling curves (Geohashes) and integrate them as routing keys within the DHT.</p> <p>- <b>Optimize Proximity Queries:</b> Develop the matching engine to query the DSTree, ensuring spatial lookups return nearest-neighbor driver nodes within a 50-millisecond threshold, utilizing Optimistic Concurrency Control to prevent double-booking.</p> |

|                                                           |                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Phase 3: Sovereign Ledger and Consensus</b></p>     | <p>Build the storage-mitigated financial settlement layer.</p>                 | <ul style="list-style-type: none"> <li>- <b>Architecture the Block-Lattice:</b> Discard global blocks in favor of asynchronous account-chains. Define the data structure for State Blocks (Send/Receive operations).</li> <li>- <b>Implement State Pruning:</b> Write the consensus rules that allow nodes to safely delete historical transactional data, maintaining only the latest Verkle Tree state roots and the corresponding IVC SNARK proofs.</li> <li>- <b>Deploy Tx-PoW:</b> Introduce localized collaborative Proof-of-Work mechanisms to throttle spam, calibrated to execute within 200ms on standard smartphone processors without thermal throttling.</li> <li>- <b>Finalize Burn Economics:</b> Implement the deflationary tokenomics engine and algorithmic dynamic pricing curve within the core protocol.</li> </ul> |
| <p><b>Phase 4: Execution Layer and State Channels</b></p> | <p>Develop the off-chain micro-transaction and smart contract environment.</p> | <ul style="list-style-type: none"> <li>- <b>Develop the On-Device zkVM:</b> Embed a lightweight zero-knowledge virtual machine into the mobile application to process smart contracts locally.</li> <li>- <b>Construct P2P State</b></li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

|  |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  | <p><b>Channels:</b> Implement the Layer-2 protocol allowing riders and drivers to lock collateral in multi-sig L1 escrows and transact instantly off-chain via Bluetooth/Wi-Fi.</p> <p>- <b>Automate Dispute Resolution:</b> Code the algorithmic slashing conditions based on the cryptographic validation of the TEE-backed Proof of Location streams.</p> <p>- <b>UX/UI Refinement:</b> Wrap the underlying cryptography, mesh networking, and blockchain node software into a frictionless mobile application that visually mimics centralized ride-sharing platforms, abstracting away the complex decentralization mechanics from the end user.</p> |
|--|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

By strictly adhering to this blueprint—merging the absolute physical isolation of hardware enclaves with the extreme data compression of recursive folding schemes, the spatial indexing power of distributed quadtrees, and the asynchronous throughput of block-lattices—it is technologically feasible to deploy a fully sovereign, infinitely scalable ride-sharing network directly onto the smartphones of its users. This architecture permanently eradicates the need for centralized cloud infrastructure, redefining the boundaries of decentralized physical infrastructure networks.

## Works cited

1. Sovereign-by-Design A Reference Architecture for AI and Blockchain Enabled Systems, accessed April 29, 2026, <https://arxiv.org/html/2602.05486v1>
2. Why Canopy Network Eliminates Layers Where Other Blockchains Still Depend on Them, accessed April 29, 2026, [https://medium.com/@sam\\_webconnect/why-canopy-network-eliminates-layers-where-other-blockchains-still-depend-on-them-61556bd2d671](https://medium.com/@sam_webconnect/why-canopy-network-eliminates-layers-where-other-blockchains-still-depend-on-them-61556bd2d671)
3. Minima - The Only Mobile-Native Layer 1 Blockchain - BelnCrypto, accessed April



29, 2026,

<https://beincrypto.com/minima-the-only-mobile-native-layer-1-blockchain/>

4. Minima: The Embedded Blockchain, accessed April 29, 2026, <https://minima.global/post/minima-the-embedded-blockchain>
5. Overview - Minima - The embedded blockchain, accessed April 29, 2026, <https://minima.global/technology/overview>
6. The Blockchain - Minima Docs, accessed April 29, 2026, <https://docs.minima.global/docs/learn/the-blockchain>
7. Minima On-Chip Blockchain Guide: Setup, Wallets, Bitget Trading, accessed April 29, 2026, <https://www.bitget.com/academy/minima-cryptocurrency-wallets-2026-guide-on-chip-blockchain-depin-mobile-security-america>
8. What Is Mina Protocol (MINA)? The World's Lightest Blockchain Explained - OneKey, accessed April 29, 2026, <https://onekey.so/blog/ecosystem/what-is-mina-protocol-mina-the-worlds-lightest-blockchain-explained/>
9. Realizing the Mina vision in Rust - ChainSafe Systems, accessed April 29, 2026, <https://blog.chainsafe.io/realizing-the-mina-vision-in-rust/>
10. Mina: Worlds Lightest Blockchain. Truly decentralized, Data Sovereign... | by hitesh mahajan | Coinmonks | Medium, accessed April 29, 2026, <https://medium.com/coinmonks/mina-worlds-lightest-blockchain-154a04cae2f3>
11. A Comprehensive Guide to Mina Protocol - DEV Community, accessed April 29, 2026, <https://dev.to/ajtech0001/a-comprehensive-guide-to-mina-protocol-4gai>
12. Mina Protocol | Mina Documentation, accessed April 29, 2026, <https://docs.minaprotocol.com/mina-protocol>
13. What are zk-SNARKs? - Mina Protocol, accessed April 29, 2026, <https://minaprotocol.com/blog/what-are-zk-snarks>
14. Mina Protocol: deep dive — Towards programmable privacy upgrade | by Obscura - Medium, accessed April 29, 2026, [https://medium.com/@obscura\\_network/mina-protocol-recursiveness-meets-succinctness-938719073d10](https://medium.com/@obscura_network/mina-protocol-recursiveness-meets-succinctness-938719073d10)
15. Mina Protocol — A Succinct Blockchain, accessed April 29, 2026, <https://minaprotocol.com/blog/coda-protocol-a-succinct-blockchain>
16. Retro: Mina Web Node Testing, accessed April 29, 2026, <https://minaprotocol.com/blog/retro-mina-web-node-testing>
17. Unveiling Mina Protocol 🛠️ - Pranshu Rastogi - Medium, accessed April 29, 2026, <https://pranshurastogi.medium.com/unveiling-mina-protocol-%EF%B8%8F-a928a45f3917>
18. What is Mina Protocol (MINA): The 'Light as a Feather' Blockchain - Explore A-Z in 2025 | henvaibta on Binance Square, accessed April 29, 2026, <https://www.binance.com/en/square/post/17504583641098>
19. Nodle Documentation, accessed April 29, 2026, <https://docs.nodle.com/>
20. MiCA Crypto-Asset White Paper - 21M Group, accessed April 29, 2026, [https://cdn.21mgroup.com/docs/mica-individual-public/NODL\\_MiCA\\_Whitepaper.pdf](https://cdn.21mgroup.com/docs/mica-individual-public/NODL_MiCA_Whitepaper.pdf)

21. Introduction - Nodle Documentation, accessed April 29, 2026,  
<https://docs.nodle.com/introduction>
22. Nodle IoT - Nodle Documentation, accessed April 29, 2026,  
<https://docs.nodle.com/nodle-iot>
23. Nodle - About - DePIN Hub, accessed April 29, 2026,  
<https://depinhub.io/projects/nodle>
24. Nodle Network White Paper v4.3, accessed April 29, 2026,  
[https://cdn.prod.website-files.com/65bb4a468049d4f4ebf2c321/66061fd2b796461e9260b006\\_whitepaper-4.3.pdf](https://cdn.prod.website-files.com/65bb4a468049d4f4ebf2c321/66061fd2b796461e9260b006_whitepaper-4.3.pdf)
25. Nodle project details | Polkadot network - Parachains.info, accessed April 29, 2026,  
<https://parachains.info/details/nodle>
26. Pi Network Complete Guide: Mobile Mining, Tokenomics, and Critical Analysis - LBank, accessed April 29, 2026,  
<https://www.lbank.com/explore/pi-network-mobile-mining-tokenomics-critical-analysis>
27. Pi Network vs. Minima, accessed April 29, 2026,  
<https://minima.global/post/pi-network-vs-minima>
28. (PDF) Enhancing Blockchain-based Ride-Sharing Services using IPFS - ResearchGate, accessed April 29, 2026,  
[https://www.researchgate.net/publication/364289250\\_Enhancing\\_Blockchain-based\\_Ride-Sharing\\_Services\\_using\\_IPFS](https://www.researchgate.net/publication/364289250_Enhancing_Blockchain-based_Ride-Sharing_Services_using_IPFS)
29. A Report by ICDE Research Fellow Morshed Mannan - Internet Archive, accessed April 29, 2026,  
[https://archive.org/download/morshed-mannan-single-web/Morshed%20Mannan\\_single\\_web.pdf](https://archive.org/download/morshed-mannan-single-web/Morshed%20Mannan_single_web.pdf)
30. Arcade City: The Future of Ridesharing Is Decentralized - FEE.org, accessed April 29, 2026,  
<https://fee.org/articles/arcade-city-the-future-of-ridesharing-is-decentralized/>
31. Solana-Based Ridesharing App Teleport Shuts Down Amid Market Challenges - Binance, accessed April 29, 2026,  
<https://www.binance.com/en/square/post/01-31-2025-solana-based-ridesharing-app-teleport-shuts-down-amid-market-challenges-19666133420042>
32. Why “Uber for X” startups failed: The supply side is king | andrewchen, accessed April 29, 2026,  
<https://andrewchen.com/why-uber-for-x-failed/>
33. (PDF) Ride-Sharing Services: From Centralization to Decentralization - ResearchGate, accessed April 29, 2026,  
[https://www.researchgate.net/publication/362049715\\_Ride-Sharing\\_Services\\_From\\_Centralization\\_to\\_Decentralization](https://www.researchgate.net/publication/362049715_Ride-Sharing_Services_From_Centralization_to_Decentralization)
34. Matthew Takara - A Decentralized Uber, Operational Analysis of Cooperative RideSourcing, A Case Study of Arcade City in Austin, Texas - doksi.net, accessed April 29, 2026,  
<https://www.doksi.net/hu/get.php?lid=30715>
35. Rechained: Sybil-Resistant Distributed Identities for the Internet of Things and Mobile Ad Hoc Networks - PMC, accessed April 29, 2026,  
<https://pmc.ncbi.nlm.nih.gov/articles/PMC8125832/>
36. Sybil Attack-Resistant Blockchain-Based Proof-of-Location Mechanism with

- Privacy Protection in VANET - PMC, accessed April 29, 2026,  
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11678958/>
37. What is Sybil Resistance in Blockchain? Understanding Sybil Attacks - Cyfrin, accessed April 29, 2026,  
<https://www.cyfrin.io/blog/understanding-sybil-attacks-in-blockchain-and-smart-contracts>
  38. Recursive Proof Systems: Compressing Entire Blockchains into One Proof | Uplatz Blog, accessed April 29, 2026,  
<https://uplatz.com/blog/recursive-proof-systems-compressing-entire-blockchain-into-one-proof/>
  39. DAG vs Blockchain - Hedera, accessed April 29, 2026,  
<https://hedera.com/learning/dag-vs-blockchain/>
  40. Block-lattice - Directed Acyclic Graphs (DAGs) - consensus, accessed April 29, 2026,  
<https://tokens-economy.gitbook.io/consensus/chain-based-dag/block-lattice-directed-acyclic-graphs-dags>
  41. Blockchain Beware: DAG And Block Lattice Are Here - Crypto Briefing, accessed April 29, 2026, <https://cryptobriefing.com/blockchain-dag-block-lattice/>
  42. (PDF) Distributed Ledger Technology: Blockchain Compared to Directed Acyclic Graph, accessed April 29, 2026,  
[https://www.researchgate.net/publication/324793344\\_Distributed\\_Ledger\\_Technology\\_Blockchain\\_Compared\\_to\\_Directed\\_Acyclic\\_Graph](https://www.researchgate.net/publication/324793344_Distributed_Ledger_Technology_Blockchain_Compared_to_Directed_Acyclic_Graph)
  43. Protocol Design - Ledger - Nano Documentation, accessed April 29, 2026,  
<https://docs.nano.org/protocol-design/ledger/>
  44. Nano How 2 : Blocks and Lattices - Medium, accessed April 29, 2026,  
<https://medium.com/nano-education/nano-how-2-blocks-and-lattices-c0ccd417bd5a>
  45. Protocol Design Introduction - Nano Documentation, accessed April 29, 2026,  
<https://docs.nano.org/protocol-design/introduction/>
  46. Support Ledger Pruning #1094 - nanocurrency/nano-node - GitHub, accessed April 29, 2026, <https://github.com/nanocurrency/nano-node/issues/1094>
  47. Verkle trees - Ethereum.org, accessed April 29, 2026,  
<https://ethereum.org/roadmap/verkle-trees/>
  48. Towards Stateless Clients in Ethereum: Benchmarking Verkle Trees and Binary Merkle Trees with SNARKs - arXiv, accessed April 29, 2026,  
<https://arxiv.org/html/2504.14069v1>
  49. Stateless Light Clients — Enabling Trustless Blockchain Interaction (3 of 9) | by Steffen Kux | Corpus.Core Insights | Medium, accessed April 29, 2026,  
<https://medium.com/corpus-core-insights/stateless-light-clients-enabling-trustless-blockchain-interaction-3-of-9-934ffd0e050c>
  50. AWS Researchers Identify Performance Bottlenecks in Ethereum's Verkle Tree Implementation - Binance, accessed April 29, 2026,  
<https://www.binance.com/en/square/post/04-02-2026-aws-researchers-identify-performance-bottlenecks-in-ethereum-s-verkle-tree-implementation-308186487512913>

51. Optimizing Merkle Proof Size Through Path Length Analysis: A Probabilistic Framework for Efficient Blockchain State Verification - MDPI, accessed April 29, 2026, <https://www.mdpi.com/1999-5903/17/2/72>
52. A Survey on the Applications of Zero-Knowledge Proofs - arXiv, accessed April 29, 2026, <https://arxiv.org/html/2408.00243v1>
53. A recursive improvement to zkRollups and zkApps for Mina - Mina Protocol, accessed April 29, 2026, <https://minaprotocol.com/blog/recursive-zkrollups-hazook>
54. microsoft/Nova: Nova: High-speed recursive zero-knowledge arguments from folding schemes - GitHub, accessed April 29, 2026, <https://github.com/microsoft/Nova>
55. Schemes for recursive proof composition - Cryptography Zoo - Anoma | Forum, accessed April 29, 2026, <https://forum.anoma.net/t/schemes-for-recursive-proof-composition/440>
56. Recursive SNARKs and incrementally verifiable computation (IVC) | Smart contract audits from Veridise, accessed April 29, 2026, <https://veridise.com/blog/learn-blockchain/recursive-snarks-and-incrementally-verifiable-computation-ivc/>
57. Intro to Nova & ZK folding schemes: Halo and accumulation - Veridise, accessed April 29, 2026, <https://veridise.com/blog/learn-blockchain/intro-to-nova-zk-folding-schemes-halo-and-accumulation/>
58. Recursive SNARKs - Berkeley RDI, accessed April 29, 2026, <https://rdi.berkeley.edu/zkp-course/assets/lecture10.pdf>
59. Intro to Nova & ZK folding schemes: Folding and nova | Smart contract audits from Veridise, accessed April 29, 2026, <https://veridise.com/blog/learn-blockchain/intro-to-nova-zk-folding-schemes-folding-and-nova/>
60. Intro to Nova & ZK folding schemes: The nuts and bolts of folding | Smart contract audits from Veridise, accessed April 29, 2026, <https://veridise.com/blog/learn-blockchain/intro-to-nova-zk-folding-schemes-the-nuts-and-bolts-of-folding/>
61. Recursive SNARKs and Incrementally Verifiable Computation (IVC) | by Veridise - Medium, accessed April 29, 2026, <https://medium.com/veridise/introduction-to-nova-and-zk-folding-schemes-4ef717574484>
62. Minima On-Chip Blockchain Guide: Setup, Wallets, Bitget Trading, accessed April 29, 2026, <https://www.bitget.com/es/academy/12560603860673>
63. What You Trust Is Insecure: Demystifying How Developers (Mis)Use Trusted Execution Environments in Practice - arXiv, accessed April 29, 2026, <https://arxiv.org/html/2512.17363v3>
64. Trusted Execution Environments (TEEs): A primer - a16z crypto, accessed April 29, 2026, <https://a16zcrypto.com/posts/article/trusted-execution-environments-tees-primer/>

65. Trusted execution environment - Wikipedia, accessed April 29, 2026, [https://en.wikipedia.org/wiki/Trusted\\_execution\\_environment](https://en.wikipedia.org/wiki/Trusted_execution_environment)
66. Integrating Blockchain Technology and OP-TEE for Enhanced Secure Computing - Medium, accessed April 29, 2026, <https://medium.com/@gwrx2005/integrating-blockchain-technology-and-op-tee-for-enhanced-secure-computing-ac1934b32f6b>
67. The Secure Enclave - Apple Support, accessed April 29, 2026, <https://support.apple.com/guide/security/the-secure-enclave-sec59b0b31ff/web>
68. Introduction to Trusted Execution Environments (TEEs) - Dynamic.xyz, accessed April 29, 2026, <https://www.dynamic.xyz/blog/trusted-execution-environments>
69. Blockchain Engineer Explains Secure Enclaves in 5 Levels of Difficulty - Medium, accessed April 29, 2026, <https://medium.com/obscuro-labs/blockchain-engineer-explains-secure-enclave-s-in-5-levels-of-difficulty-a9f4b5e12dfd>
70. Trusted Execution Environment (TEE) - Ledger, accessed April 29, 2026, <https://www.ledger.com/academy/glossary/trusted-execution-environment-tee>
71. Blockchain Based Zero-Knowledge Proof of Location in IoT - arXiv, accessed April 29, 2026, <https://arxiv.org/html/2406.18389v1>
72. Sybil Attack-Resistant Blockchain-Based Proof-of-Location Mechanism with Privacy Protection in VANET - MDPI, accessed April 29, 2026, <https://www.mdpi.com/1424-8220/24/24/8140>
73. Proving software integrity using Trusted Execution Environment/ Secure enclave - Reddit, accessed April 29, 2026, [https://www.reddit.com/r/cryptography/comments/sfsgk1/proving\\_software\\_integrity\\_using\\_trusted/](https://www.reddit.com/r/cryptography/comments/sfsgk1/proving_software_integrity_using_trusted/)
74. Sybil Attack-Resistant Blockchain-Based Proof-of-Location Mechanism with Privacy Protection in VANET - PubMed, accessed April 29, 2026, <https://pubmed.ncbi.nlm.nih.gov/39771874/>
75. Top Blockchain Hardware Solutions - AJProTech, accessed April 29, 2026, <https://ajprotech.com/blog/insights/hardware-for-blockchain.html>
76. Trusted Execution Environments (TEEs): A Comprehensive Guide : r/techUniverse - Reddit, accessed April 29, 2026, [https://www.reddit.com/r/techUniverse/comments/1s7dak8/trusted\\_execution\\_environments\\_tees\\_a/](https://www.reddit.com/r/techUniverse/comments/1s7dak8/trusted_execution_environments_tees_a/)
77. Designing Uber: Geospatial Indexing, WebSockets, and Distributed Locks - DEV Community, accessed April 29, 2026, [https://dev.to/ganesh\\_parella/designing-uber-geospatial-indexing-websockets-and-distributed-locks-4mhb](https://dev.to/ganesh_parella/designing-uber-geospatial-indexing-websockets-and-distributed-locks-4mhb)
78. Quad Trees | System Design - AlgoMaster.io, accessed April 29, 2026, <https://algomaster.io/learn/system-design/quad-tree>
79. (PDF) Peer-to-Peer Shared Ride Systems - ResearchGate, accessed April 29, 2026, [https://www.researchgate.net/publication/221118240\\_Peer-to-Peer\\_Shared\\_Ride\\_Systems](https://www.researchgate.net/publication/221118240_Peer-to-Peer_Shared_Ride_Systems)
80. FRAMEWORK OF QUERY INDEXING SPATIAL DATA IN P2P SYSTEMS - iosrjen.org.,



- accessed April 29, 2026, [https://iosrjen.org/Papers/vol2\\_issue3/O023451453.pdf](https://iosrjen.org/Papers/vol2_issue3/O023451453.pdf)
81. BLE or Wi-Fi: what you need to know - ELA Innovation, accessed April 29, 2026, <https://elainnovation.com/en/ble-vs-wi-fi-what-you-need-to-know/>
  82. Wi-Fi Aware or Bluetooth Low Energy (BLE) for mobile mesh | by Micha Anthenor Benoliel, accessed April 29, 2026, <https://anthenor.medium.com/wi-fi-aware-or-bluetooth-low-energy-ble-for-mobile-mesh-ea11defcfd46>
  83. Wireless Connectivity Technology Selection Guide - Texas Instruments, accessed April 29, 2026, <https://www.ti.com/lit/swat016>
  84. Design a Ride-Sharing Platform Like Uber - System Design Handbook, accessed April 29, 2026, <https://www.systemdesignhandbook.com/guides/design-uber/>
  85. Distributed hash table - Wikipedia, accessed April 29, 2026, [https://en.wikipedia.org/wiki/Distributed\\_hash\\_table](https://en.wikipedia.org/wiki/Distributed_hash_table)
  86. Peer-to-Peer Systems and Distributed Hash Tables - cs.Princeton, accessed April 29, 2026, <https://www.cs.princeton.edu/courses/archive/fall18/cos418/docs/L6-dhts.pdf>
  87. Using a Distributed Quadtree Index in Peer-to-Peer Networks, accessed April 29, 2026, <https://people.eng.unimelb.edu.au/etanin/vldbj07.pdf>
  88. DSTree: A Spatio-Temporal Indexing Data Structure for Distributed Networks - MDPI, accessed April 29, 2026, <https://www.mdpi.com/2297-8747/29/3/42>
  89. Spatial Indexing with QuadTrees - Medium, accessed April 29, 2026, <https://medium.com/@waleoyediran/spatial-indexing-with-quadtrees-b998ae49336>
  90. A QUADTREE SPATIAL INDEX METHOD WITH INCLUSION RELATIONS FOR THE INCREMENTAL UPDATING OF VECTOR LANDCOVER DATABASE, accessed April 29, 2026, <https://isprs-archives.copernicus.org/articles/XLII-4/757/2018/>
  91. System Design: Design a Geo-Spatial index for real-time location search | by Kousik Nath, accessed April 29, 2026, <https://kousiknath.medium.com/system-design-design-a-geo-spatial-index-for-real-time-location-search-10968fe62b9c>
  92. Distributed Hash Tables (DHTs) - Medium, accessed April 29, 2026, <https://medium.com/@aleenat.csa2024/distributed-hash-tables-dhts-8546eef61f07>
  93. How much better is using quad trees than simple relational database for storing location data?, accessed April 29, 2026, <https://softwareengineering.stackexchange.com/questions/454982/how-much-better-is-using-quad-trees-than-simple-relational-database-for-storing>
  94. The Evolution of BlockDAG Technology: From Concept to Distributed Ledger Innovation, accessed April 29, 2026, <https://qitmeer.medium.com/the-evolution-of-blockdag-technology-from-concept-to-distributed-ledger-innovation-e47d098ca66e>
  95. Possible futures of the Ethereum protocol, part 4: The Verge - eth.limo, accessed April 29, 2026, <https://vitalik.eth.limo/general/2024/10/23/futures4.html>
  96. GENES: An Efficient Recursive zk-SNARK and Its Novel Application in Blockchain - MDPI, accessed April 29, 2026, <https://www.mdpi.com/2079-9292/14/3/492>

97. The Optimization Model of Ride-Sharing Route for Ride Hailing Considering Both System Optimization and User Fairness - MDPI, accessed April 29, 2026, <https://www.mdpi.com/2071-1050/13/2/902>
98. A Distributed Model-Free Ride-Sharing Approach for Joint Matching, Pricing, and Dispatching Using Deep Reinforcement Learning - CS@Purdue, accessed April 29, 2026, <https://www.cs.purdue.edu/homes/bb/ridesharing.pdf>
99. ZK Tech You Should Know — Part 1: SNARKs & STARKs | Mina Protocol, accessed April 29, 2026, <https://minaprotocol.com/blog/zk-you-should-know-snarks-starks>
100. CAPPR-Wallet: a context-aware and recoverable wallet architecture with privacy-preserving rules for trustless blockchain ecosystems - PMC, accessed April 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC13106698/>